

# Running surface-partition on Pelle

## A Step-by-Step Guide for New Collaborators

May 2026

### Abstract

This guide covers everything a new collaborator needs to go from zero to running **surface-partition** jobs on UPPMAX Pelle: obtaining cluster access, setting up the Python environment, cloning and configuring the repository, submitting Phase 1 relaxation jobs and Phase 2 refinement jobs, running parameter sweeps, and retrieving results for local analysis. No prior experience with Pelle or SLURM is assumed.

## Contents

|          |                                                    |           |
|----------|----------------------------------------------------|-----------|
| <b>1</b> | <b>Prerequisites</b>                               | <b>3</b>  |
| <b>2</b> | <b>Logging in to Pelle</b>                         | <b>3</b>  |
| <b>3</b> | <b>Directory Layout on Pelle</b>                   | <b>3</b>  |
| <b>4</b> | <b>First-Time Setup</b>                            | <b>4</b>  |
| 4.1      | Create the directory structure . . . . .           | 4         |
| 4.2      | Clone the repository . . . . .                     | 4         |
| 4.3      | Load the Python module . . . . .                   | 4         |
| 4.4      | Create the Python virtual environment . . . . .    | 4         |
| 4.5      | Install the project and its dependencies . . . . . | 5         |
| 4.6      | Verify the installation . . . . .                  | 5         |
| 4.7      | Configure the submission scripts . . . . .         | 5         |
| 4.8      | Create the SLURM log directory . . . . .           | 5         |
| <b>5</b> | <b>Experiment Configuration Files</b>              | <b>6</b>  |
| 5.1      | File naming convention . . . . .                   | 6         |
| 5.2      | Top-level structure . . . . .                      | 6         |
| 5.3      | Phase 1 relaxation parameters . . . . .            | 7         |
| 5.4      | Surface geometry parameters . . . . .              | 7         |
| <b>6</b> | <b>Running Phase 1: Relaxation</b>                 | <b>8</b>  |
| 6.1      | How the submission script works . . . . .          | 8         |
| 6.2      | Basic usage . . . . .                              | 8         |
| 6.3      | Available options . . . . .                        | 9         |
| 6.4      | Example commands . . . . .                         | 9         |
| 6.5      | Monitoring your job . . . . .                      | 9         |
| 6.6      | Reading the job output . . . . .                   | 10        |
| <b>7</b> | <b>Running Phase 2: Perimeter Refinement</b>       | <b>10</b> |

|           |                                                                   |           |
|-----------|-------------------------------------------------------------------|-----------|
| 7.1       | Locating the Phase 1 solution . . . . .                           | 10        |
| 7.2       | Basic usage . . . . .                                             | 10        |
| 7.3       | Available options . . . . .                                       | 10        |
| 7.4       | Example commands . . . . .                                        | 11        |
| 7.5       | Phase 2 refinement parameters . . . . .                           | 11        |
| 7.6       | Timing profiling . . . . .                                        | 12        |
| <b>8</b>  | <b>Parameter Sweeps</b>                                           | <b>13</b> |
| 8.1       | Sweep specification files . . . . .                               | 13        |
| 8.2       | Sweep YAML structure . . . . .                                    | 13        |
| 8.3       | Submitting a sweep . . . . .                                      | 14        |
| 8.4       | Collecting results manually . . . . .                             | 14        |
| <b>9</b>  | <b>Output Directory Structure</b>                                 | <b>15</b> |
| 9.1       | Home directory (SLURM logs) . . . . .                             | 15        |
| 9.2       | Shared project directory (run results) . . . . .                  | 15        |
| 9.3       | Per-run directory layout . . . . .                                | 15        |
| 9.4       | Experiment directory layout (parameter sweeps) . . . . .          | 15        |
| <b>10</b> | <b>Managing Disk Space</b>                                        | <b>16</b> |
| 10.1      | Checking current disk usage . . . . .                             | 16        |
| 10.2      | Pruning runs with <code>cleanup_sweep_results.py</code> . . . . . | 16        |
| 10.2.1    | Typical workflow . . . . .                                        | 17        |
| 10.2.2    | Available options . . . . .                                       | 17        |
| 10.2.3    | Further examples . . . . .                                        | 17        |
| <b>11</b> | <b>Retrieving and Analysing Results</b>                           | <b>17</b> |
| 11.1      | Transferring files to your local machine . . . . .                | 17        |
| 11.2      | Local analysis (on your laptop) . . . . .                         | 18        |
| 11.3      | Visualising partitions (requires PyVista) . . . . .               | 18        |
| <b>12</b> | <b>Useful SLURM Commands</b>                                      | <b>19</b> |
| <b>13</b> | <b>Troubleshooting</b>                                            | <b>19</b> |
| <b>A</b>  | <b>pelle_config.sh Quick Reference</b>                            | <b>20</b> |
| <b>B</b>  | <b>Available Experiment Configurations</b>                        | <b>20</b> |

## 1 Prerequisites

Before starting, make sure you have the following:

- **UPPMAX account.** Request one via the SUPR portal (<https://supr.naiss.se>). Ask the project PI to add you to the allocation `uppmx2025-2-534` once your account is active.
- **SSH client.** On Linux and macOS the built-in `ssh` command is sufficient. On Windows, use Git Bash, WSL, or PuTTY.
- **Repository access.** You need read access to the GitHub repository. Ask the PI for the URL and confirm you can clone it before your first login to Pelle.
- **Basic command-line familiarity.** You should be comfortable navigating directories, editing text files, and running commands in a Linux shell. No prior HPC or SLURM experience is required.

## 2 Logging in to Pelle

Pelle is UPPMAX's newest cluster. Connect with SSH, replacing `<your_username>` with your UPPMAX username:

```
ssh <your_username>@pelle.uppmx.uu.se
```

The first connection will ask you to confirm the host fingerprint — type **yes** and press Enter.

### Tip

**SSH key authentication.** Password prompts can get tedious. Set up key-based login once:

```
# On your local machine
ssh-keygen -t ed25519 -C "pelle"
ssh-copy-id <your_username>@pelle.uppmx.uu.se
```

After this, `ssh pelle.uppmx.uu.se` logs you in without a password.

## 3 Directory Layout on Pelle

UPPMAX uses a two-tier storage model. Understanding it now prevents confusion later about where files end up.

| Location                                   | Quota | Notes                                                    |
|--------------------------------------------|-------|----------------------------------------------------------|
| <code>~/</code>                            | 32 GB | Home directory. Backed up. Personal to each user.        |
| <code>/proj/snic2020-15-36/private/</code> | Large | Project directory. <b>Shared</b> by all project members. |

The conventions used by this project are:

- **Code, configuration, scripts** — live in your home directory under `~/projects/surface-partition/`. Each collaborator has their own copy.
- **Python virtual environment** — lives in your home directory under `~/venvs/surface-partition/`. Each collaborator has their own environment.
- **Computation results** (HDF5 solution files, refinement checkpoints, traces) — written to the shared project directory at `/proj/snic2020-15-36/private/LINKED_LST_MANIFOLD/results/`. All collaborators share this directory.

- **SLURM log files** — written inside the repository at `~/projects/surface-partition/slurm_logs/`. Personal to each user.

#### Important

The shared project directory `/proj/snic2020-15-36/private/` is **not backed up**. Do not use it as the only copy of important data. Transfer final results to your local machine or a backed-up location.

## 4 First-Time Setup

Perform these steps once on your first login. They take about ten minutes.

### 4.1 Create the directory structure

```
mkdir -p ~/projects
mkdir -p ~/venvs
```

### 4.2 Clone the repository

Clone the repository from GitHub:

```
cd ~/projects
git clone git@github.com:TebanVe/surface-partition.git surface-partition
```

#### Note

This uses SSH-based cloning, which requires your SSH public key to be registered on GitHub. If you have not done this yet, go to <https://github.com/settings/keys> and add the public key from Pelle (`~/.ssh/id_ed25519.pub` or similar). Alternatively, use the HTTPS URL `https://github.com/TebanVe/surface-partition.git` and authenticate with a personal access token.

This creates `~/projects/surface-partition/` with the full repository contents.

### 4.3 Load the Python module

Pelle uses the *module* system to manage software. Load Python with:

```
module load Python/3.11.5-GCCcore-13.3.0
```

#### Note

To see which Python versions are available: `module spider Python`. The project currently targets `Python/3.11.5-GCCcore-13.3.0`. If it is unavailable, contact the PI for an updated module name and update `cluster/pelle_config.sh` accordingly.

### 4.4 Create the Python virtual environment

```
python -m venv ~/venvs/surface-partition
source ~/venvs/surface-partition/bin/activate
pip install --upgrade pip
```

## 4.5 Install the project and its dependencies

```
cd ~/projects/surface-partition
pip install -e ".[viz,implicit]"
```

This installs:

- **Core:** numpy, scipy, pyyaml, matplotlib, h5py, tqdm
- **viz:** pyvista (3D visualisation — used by scripts with `-save` for headless image output)
- **implicit:** scikit-image (marching-cubes meshing for double-torus and Banchoff-Chmutov surfaces)

### Important

**IPOPT is not available on Pelle.** The `cyipopt` package requires a system-level IPOPT installation which is not present on UPPMAX. Phase 2 refinement on the cluster uses `-method slsqp` (default) or `-method trust-constr`. The full IPOPT workflow (including exact-Hessian mode) is available on your local machine once the results are transferred.

## 4.6 Verify the installation

```
python -c "import numpy, scipy, h5py, yaml, pyvista; print('OK')"
```

If this prints OK without errors, the environment is ready.

## 4.7 Configure the submission scripts

The file `cluster/pelle_config.sh` is the single place where user-specific paths are set. Open it with your favourite editor:

```
nano ~/projects/surface-partition/cluster/pelle_config.sh
```

Find and update these two lines, replacing `teban66` with your own UPPMAX username:

```
REPO_DIR="/home/<your_username>/projects/surface-partition"
VENV_DIR="/home/<your_username>/venvs/surface-partition"
```

Leave all other lines unchanged — they refer to the shared project allocation and storage paths that are identical for every collaborator.

### Tip

After editing `pelle_config.sh`, prevent accidental commits of your personal paths by running once:

```
cd ~/projects/surface-partition
git update-index --assume-unchanged cluster/pelle_config.sh
```

Git will then ignore local changes to that file when staging commits.

## 4.8 Create the SLURM log directory

The submission scripts write job output here. Create it once:

```
mkdir -p ~/projects/surface-partition/slurm_logs
```

## 5 Experiment Configuration Files

Every experiment is driven by a single YAML file that configures both Phase 1 and Phase 2. The `parameters/` directory ships four ready-to-use configs (Appendix B). For a new experiment, copy the closest one and adapt it.

### 5.1 File naming convention

Config files follow the pattern `{surface}_{npart}part.yaml`. Examples from the repository:

| File                                      | Experiment                              |
|-------------------------------------------|-----------------------------------------|
| <code>torus_10part.yaml</code>            | Torus, 10 partitions                    |
| <code>ellipsoid_6part.yaml</code>         | Ellipsoid, 6 partitions                 |
| <code>double_torus_10part.yaml</code>     | Double torus, 10 partitions             |
| <code>banchoff_chmutov_12part.yaml</code> | Banchoff-Chmutov order 4, 12 partitions |

#### Tip

To start a new experiment, copy the config whose surface type matches yours and rename it following the convention above:

```
cp parameters/torus_10part.yaml parameters/torus_6part.yaml
nano parameters/torus_6part.yaml # or your preferred editor
```

### 5.2 Top-level structure

Every config has four top-level sections:

```
experiment:      # run identity
  name: torus-10-partitions
  surface: torus  # surface identifier (see table below)

relaxation:      # Phase 1: Gamma-convergence PGD
  n_partitions: 10
  lambda_penalty: 3.25
  seed: 13001502
  ...

surface:         # mesh geometry, keyed by the surface identifier
  torus:
    n_theta: 80
    n_phi: 66
    R: 1.0
    r: 0.6
    ...

refinement:      # Phase 2: perimeter optimizer
  method: slsqp
  max_iterations: 8
  ...
```

The value of `experiment.surface` must match the key used inside the `surface:` block. Valid identifiers are:

| Identifier                    | Surface                                            |
|-------------------------------|----------------------------------------------------|
| <code>torus</code>            | Parametric torus of revolution                     |
| <code>ellipsoid</code>        | Parametric ellipsoid                               |
| <code>double_torus</code>     | Implicit double torus (marching cubes)             |
| <code>banchoff_chmutov</code> | Implicit Banchoff-Chmutov order 4 (marching cubes) |

### 5.3 Phase 1 relaxation parameters

The `relaxation:` block controls the Projected Gradient Descent optimiser. The parameters most commonly tuned when setting up a new experiment are:

| Parameter                      | Typical value     | Description                                                                                                                                                                                                                                                                              |
|--------------------------------|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>n_partitions</code>      | 6–12              | Number of regions to partition the surface into.                                                                                                                                                                                                                                         |
| <code>lambda_penalty</code>    | 2.0–5.0           | Weight of the $\Gamma$ -convergence penalty term. <b>Most impactful parameter.</b> Too small $\Rightarrow$ fuzzy, overlapping boundaries. Too large $\Rightarrow$ stalled convergence. Start from the nearest existing config; use a parameter sweep (Section 8) to find the best value. |
| <code>seed</code>              | any integer       | Random seed for the initial density field. Different seeds reach different local optima. Running several seeds in a sweep is the standard way to find a good partition.                                                                                                                  |
| <code>max_iter</code>          | 15 000–<br>25 000 | Maximum PGD iterations per refinement level. Increase if the energy is still decreasing when the run ends.                                                                                                                                                                               |
| <code>refinement_levels</code> | 2–5               | Number of multi-level mesh refinement steps. Each step adds the surface increment values to the current resolution. More levels produce finer boundaries at the cost of longer runtimes.                                                                                                 |

#### Note

The remaining `relaxation:` parameters (`pgd_step0`, `pgd_armijo_c`, `penalty_target_mode`, and the `refine_trigger_*` thresholds) control low-level PGD behaviour. The shipped defaults are stable across all surfaces and rarely need to be changed.

### 5.4 Surface geometry parameters

The `surface:` block specifies the mesh geometry.

**Parametric surfaces** (`torus`, `ellipsoid`) define a base-level grid resolution and an increment added at each refinement level:

| Surface   | Key parameters                                                                                                                                                | Notes                                                                                                                                          |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| torus     | <code>n_theta</code> , <code>n_phi</code> (base)<br><code>R</code> , <code>r</code> (radii)<br><code>n_theta_increment</code><br><code>n_phi_increment</code> | <code>n_theta</code> : samples along the major circle. <code>n_phi</code> : samples along the tube. Increments are added per refinement level. |
| ellipsoid | <code>n_theta</code> , <code>n_phi</code> (base)<br><code>a</code> , <code>b</code> , <code>c</code> (semi-axes)<br>increments                                | The shipped config uses $a = b = 1$ , $c = 0.7$ (oblate spheroid). Adjust the semi-axes to change the shape.                                   |

**Implicit surfaces** (`double_torus`, `bachhoff_chmutov`) use a 3-D Cartesian grid for marching-cubes meshing:

| Surface                       | Key parameters                                                                                                                                                                           | Notes                                                                                                                                                                                       |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>double_torus</code>     | <code>n_grid_x</code> , <code>n_grid_y</code> ,<br><code>n_grid_z</code> (base)<br><code>c</code> (tube thickness)<br><code>n_grid_x_increment</code><br><code>n_grid_y_increment</code> | Default $c=0.03$ . Only $x$ and $y$ increments are used; $z$ scales proportionally. Maintain the ratio $x : y : z \approx 8 : 6 : 3$ from the shipped config to avoid degenerate triangles. |
| <code>bachhoff_chmutov</code> | <code>n_grid_x</code> , <code>n_grid_y</code> ,<br><code>n_grid_z</code> (base)<br><code>n_grid_x_increment</code><br><code>n_grid_y_increment</code>                                    | The largest connected component is kept automatically. Scale all three base dimensions proportionally when changing resolution.                                                             |

## 6 Running Phase 1: Relaxation

Phase 1 performs Gamma-convergence relaxation (Projected Gradient Descent) on the surface mesh. It produces an HDF5 solution file used as input for Phase 2.

### 6.1 How the submission script works

`cluster/submit_relaxation.sh` generates a SLURM job script on the fly from the parameters in `cluster/pelle_config.sh` and the experiment YAML you provide, then submits it with `sbatch`. The running job:

1. Loads the Python module and activates the virtual environment.
2. Changes to `PROJECT_BASE` so results land in the shared directory.
3. Runs `scripts/find_surface_partition.py` with your config.

Results are written to `/proj/snic2020-15-36/private/LINKED_LST_MANIFOLD/results/`.

### 6.2 Basic usage

From inside the repository directory:

```
cd ~/projects/surface-partition
bash cluster/submit_relaxation.sh --config parameters/torus_10part.yaml
```



### 6.3 Available options

| Option              | Default    | Description                           |
|---------------------|------------|---------------------------------------|
| -config <yaml>      | (required) | Experiment YAML file                  |
| -time <HH:MM:SS>    | 12:00:00   | Wall-clock time limit                 |
| -cpus <N>           | 4          | Number of CPU cores                   |
| -mem <XG>           | 16G        | Memory per job                        |
| -resume-from <h5>   | —          | Warm-start from a prior solution      |
| -solution-dir <dir> | —          | Override output directory             |
| -dry-run            | —          | Print SLURM script without submitting |

### 6.4 Example commands

Torus, 10 partitions (default config):

```
bash cluster/submit_relaxation.sh --config parameters/torus_10part.yaml
```

Ellipsoid, 6 partitions with longer time limit:

```
bash cluster/submit_relaxation.sh \
  --config parameters/ellipsoid_6part.yaml \
  --time 24:00:00 --cpus 8
```

Double torus, 10 partitions (implicit surface):

```
bash cluster/submit_relaxation.sh \
  --config parameters/double_torus_10part.yaml
```

Warm-start from a prior solution at a higher refinement level:

```
bash cluster/submit_relaxation.sh \
  --config parameters/torus_10part.yaml \
  --resume-from /proj/snic2020-15-36/private/LINKED_LST_MANIFOLD/results/ \
  run_20260510_120000_.../solution/surface_...h5
```

Preview the SLURM script without submitting:

```
bash cluster/submit_relaxation.sh \
  --config parameters/torus_10part.yaml --dry-run
```

### 6.5 Monitoring your job

```
# List all your running and queued jobs
squeue -u <your_username>

# Watch the queue (refreshes every 5 seconds)
watch -n 5 squeue -u <your_username>

# Cancel a job
scancel <job_id>

# Show detailed accounting for a finished job
sacct -j <job_id> --format=JobID,Elapsed,CPUTime,MaxRSS,State

# UPPMAX convenience tool for detailed job info
jobinfo <job_id>
```

## 6.6 Reading the job output

SLURM captures `stdout` and `stderr` in the log directory:

```
ls ~/projects/surface-partition/slurm_logs/
# Files are named: <job_name>_<job_id>.out and <job_name>_<job_id>.err

# Follow the live output of a running job
tail -f ~/projects/surface-partition/slurm_logs/<job_name>_<job_id>.out
```

## 7 Running Phase 2: Perimeter Refinement

Phase 2 takes the HDF5 solution from Phase 1 and minimises the total boundary perimeter subject to equal-area constraints. It requires a completed Phase 1 run.

### 7.1 Locating the Phase 1 solution

After Phase 1 finishes, find the solution file:

```
ls /proj/snic2020-15-36/private/LINKED_LST_MANIFOLD/results/
# Each run is in a directory named:
# run_<timestamp>_surf<surface>_npart<N>_..._lam<lambda>_seed<seed>/

ls /proj/snic2020-15-36/private/LINKED_LST_MANIFOLD/results/ \
  run_<your_run>/solution/
# The HDF5 file is: surface_part<N>_surf<surface>_..._<timestamp>.h5
```

### 7.2 Basic usage

```
bash cluster/submit_refinement.sh \
  --solution /proj/snic2020-15-36/private/LINKED_LST_MANIFOLD/results/ \
    run_<your_run>/solution/surface_...h5 \
  --config parameters/torus_10part.yaml
```

### 7.3 Available options

| Option                                 | Default    | Description                                             |
|----------------------------------------|------------|---------------------------------------------------------|
| <code>-solution &lt;h5&gt;</code>      | (required) | Phase 1 solution or checkpoint                          |
| <code>-config &lt;yaml&gt;</code>      | (required) | Experiment YAML                                         |
| <code>-time &lt;HH:MM:SS&gt;</code>    | 24:00:00   | Wall-clock time limit                                   |
| <code>-cpus &lt;N&gt;</code>           | 4          | CPU cores                                               |
| <code>-mem &lt;XG&gt;</code>           | 16G        | Memory per job                                          |
| <code>-method &lt;name&gt;</code>      | slsqp      | Solver: <code>slsqp</code> or <code>trust-constr</code> |
| <code>-max-iterations &lt;N&gt;</code> | from YAML  | Migration loop iterations                               |
| <code>-max-opt-iter &lt;N&gt;</code>   | from YAML  | Solver iterations per loop step                         |
| <code>-tolerance &lt;val&gt;</code>    | from YAML  | Solver convergence tolerance                            |
| <code>-boundary-tol &lt;val&gt;</code> | from YAML  | Type 1 migration threshold                              |
| <code>-lbfgs-memory &lt;N&gt;</code>   | from YAML  | L-BFGS history length                                   |
| <code>-save-iterations</code>          | —          | Save every intermediate checkpoint                      |
| <code>-best-iterate</code>             | —          | Keep best-perimeter iterate                             |
| <code>-allow-partial</code>            | —          | Continue past constraint violations                     |
| <code>-dry-run</code>                  | —          | Print script without submitting                         |

## 7.4 Example commands

Standard run using config defaults (SLSQP):

```
bash cluster/submit_refinement.sh \  
  --solution /proj/.../results/run_.../solution/surface_....h5 \  
  --config parameters/torus_10part.yaml
```

Trust-region constrained solver, 20 iterations:

```
bash cluster/submit_refinement.sh \  
  --solution /proj/.../results/run_.../solution/surface_....h5 \  
  --config parameters/torus_10part.yaml \  
  --method trust-constr --max-iterations 20
```

Resume from a checkpoint (auto-detected by the pipeline):

```
bash cluster/submit_refinement.sh \  
  --solution /proj/.../results/run_.../refinement/slsqp_btol0.001/ \  
    iteration_003_20260510_140000.h5 \  
  --config parameters/torus_10part.yaml
```

### Note

Phase 2 refinement campaigns are stored under `results/run.../refinement/` inside a subdirectory named by solver and tolerances, for example `slsqp_btol0.001/` or `trust-constr_btol0.001_lbfgs30/`. A `refinement.yaml` config snapshot and `refinement.log` are written inside each campaign directory.

## 7.5 Phase 2 refinement parameters

The `refinement:` block in the experiment YAML configures the perimeter minimiser. CLI flags passed to `submit_refinement.sh` override YAML values for individual runs.

| Parameter                 | Values / range                                  | Description                                                                                                                                                                                                                                                                          |
|---------------------------|-------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| method                    | <code>slsqp</code><br><code>trust-constr</code> | Solver to use on Pelle. <code>slsqp</code> is faster and the recommended default; <code>trust-constr</code> can be more robust on difficult topologies. ( <code>ipopt</code> requires a system-level installation not present on UPPMAX; use it locally after transferring results.) |
| max_iterations            | 2–10                                            | Number of topology cycles (optimize → detect migrations → migrate). Increase if migrations are still occurring at the last cycle.                                                                                                                                                    |
| max_opt_iter              | 200–1 000                                       | Solver calls per topology cycle. Increase if the perimeter is still decreasing when the optimizer terminates.                                                                                                                                                                        |
| tolerance                 | 1e-6–1e-7                                       | Solver convergence tolerance. Tighter values give more precision at the cost of longer runtimes.                                                                                                                                                                                     |
| boundary_tol              | 0.001–0.01                                      | Type 1 migration threshold: a variable point whose parameter $\lambda$ is within this distance of 0 or 1 triggers a vertex-collapse migration.                                                                                                                                       |
| lbfgs_memory              | 20–50                                           | L-BFGS history length. Larger values can improve convergence on well-conditioned problems but use more memory.                                                                                                                                                                       |
| save_iterations           | true/false                                      | Save an HDF5 checkpoint after every topology cycle. Strongly recommended <b>true</b> on the cluster so any cycle can be resumed from the last checkpoint.                                                                                                                            |
| allow_partial_convergence | true/false                                      | If <b>true</b> , proceed to migration detection even if the solver hit <code>max_opt_iter</code> without converging. Useful for long runs where some progress per cycle is better than none.                                                                                         |

#### Note

`exact_hessian` and `best_iterate` are IPOPT-specific options that are silently ignored by `slsqp` and `trust-constr`.

## 7.6 Timing profiling

Add `-profile` to any `submit_refinement.sh` call to record per-callback wall-clock times during Phase 2:

```
bash cluster/submit_refinement.sh \
  --solution /proj/.../results/run_.../solution/surface_...h5 \
  --config parameters/torus_10part.yaml \
  --profile
```

When profiling is active, a `timing_profile.yaml` file is written inside the campaign directory after each run. It records:

- Total wall-clock time and total iteration count.
- Per-callback percentage breakdown: objective evaluation, gradient evaluation, Steiner recomputation, and other callbacks.
- Raw Steiner recomputation counts.

After a sweep where `-profile` was used, transfer results locally and run the timing analyser to generate scaling figures:

```
python sweep/timing_analyzer.py \
    --experiment-dir ~/local-results/torus_npart10/
```

### Tip

Profiling uses standard-library timers only and adds negligible overhead. It can safely be left on for all cluster runs without affecting runtime.

## 8 Parameter Sweeps

The sweep tool submits a grid (or paired) array of Phase 1 jobs in one command, collects results into a shared index, and produces analysis plots.

### 8.1 Sweep specification files

Sweep specs live under `sweep/parameters/`. Each YAML file defines the surface, partitions, and the parameters to vary. Examples:

- `sweep/parameters/sweep_torus_lambda.yaml` — grid sweep over  $\lambda$  values and random seeds for the torus.
- `sweep/parameters/sweep_double_torus_lambda.yaml` — grouped grid sweep over  $\lambda$  and mesh resolution.

### 8.2 Sweep YAML structure

A sweep file has four top-level keys:

```
name: lambda-seed          # short label; appears in output filenames
base_config: parameters/torus_10part.yaml # path from repo root

sweep:
  relaxation.lambda_penalty: [2.5, 3.0, 3.25, 3.5, 4.0]
  relaxation.seed: [13131313, 84172851, 52698790]
  relaxation.refinement_levels: [5]

strategy: grid            # "grid" (Cartesian product) or "paired" (zip)
```

Keys inside `sweep:` use **dot notation** to address nested keys from the base config: `relaxation.lambda_penalty` sets `lambda_penalty` inside the `relaxation:` block. Each value is a list; the strategy determines how the lists are combined:

- **strategy:** `grid` — Cartesian product of all lists. The example above produces  $5 \times 3 \times 1 = 15$  runs.
- **strategy:** `paired` — zips lists together element-by-element (all lists must be the same length). Produces one run per element.

**Grouped parameters.** Parameters that must always scale together — such as the  $x$  and  $y$  grid dimensions of an implicit surface — are placed in a named group. The group’s lists are zipped internally; the group then participates in the outer Cartesian product as a single unit:

```
sweep:
  relaxation.lambda_penalty: [1.0, 2.0, 3.0]
  resolution:                # named group -- lists are zipped
    surface.double_torus.n_grid_x: [54, 64, 74]
    surface.double_torus.n_grid_y: [40, 50, 60]

strategy: grid    # 3 lambdas x 3 resolution pairs = 9 runs
```

**Naming convention for sweep files:** `sweep_{surface}_{what}.yaml`, placed under `sweep/parameters/`. For example, `sweep_torus_lambda.yaml` sweeps  $\lambda$  and seed on the torus; `sweep_double_torus_lambda.yaml` sweeps  $\lambda$  and resolution on the double torus.

### Tip

To create a new sweep, copy the closest existing sweep file and update `name`, `base_config`, and the `sweep:` block. Run with `-dry-run` first to confirm the number and parameters of the generated runs before submitting.

## 8.3 Submitting a sweep

```
# Preview: see how many jobs will be submitted (no jobs submitted)
bash cluster/submit_sweep.sh \
  --sweep sweep/parameters/sweep_torus_lambda.yaml --dry-run

# Submit all combinations as separate SLURM jobs
bash cluster/submit_sweep.sh \
  --sweep sweep/parameters/sweep_torus_lambda.yaml

# Submit with a 24-hour time limit and auto-collect when done
bash cluster/submit_sweep.sh \
  --sweep sweep/parameters/sweep_torus_lambda.yaml \
  --time 24:00:00 --auto-collect
```

When `-auto-collect` is given, a final SLURM job is submitted with a dependency on all sweep jobs. It runs automatically after the last run finishes, scans the results, and writes `experiment_index.yaml` plus summary CSV files.

## 8.4 Collecting results manually

If you did not use `-auto-collect`, run the collector once all jobs have finished:

```
# Recommended: index + matplotlib analysis plots (no PyVista screenshots)
python sweep/parameter_sweep.py \
  --sweep sweep/parameters/sweep_torus_lambda.yaml \
  --mode collect --collect-skip-screenshots \
  --output-dir /proj/snrc2020-15-36/private/LINKED_LST_MANIFOLD/results

# Index and CSV only (fastest):
python sweep/parameter_sweep.py \
  --sweep sweep/parameters/sweep_torus_lambda.yaml \
  --mode collect --collect-metadata-only \
  --output-dir /proj/snrc2020-15-36/private/LINKED_LST_MANIFOLD/results
```

**Note**

The collector must be run from inside the repository:

```
cd ~/projects/surface-partition
source ~/venvs/surface-partition/bin/activate
module load Python/3.11.5-GCCcore-13.3.0
```

You can run the collector interactively on a login node for short sweeps; for sweeps with many runs or large meshes, submit it as a job instead.

## 9 Output Directory Structure

### 9.1 Home directory (SLURM logs)

```
~/projects/surface-partition/
+-- slurm_logs/
|   |-- relax_torus_npart10_..._<job_id>.out
|   |-- relax_torus_npart10_..._<job_id>.err
+-- ...
```

These files contain the console output of each SLURM job. The `.out` file mirrors the Python logger output; `.err` captures unexpected errors.

### 9.2 Shared project directory (run results)

All computation results go here:

```
/proj/snic2020-15-36/private/LINKED_LST_MANIFOLD/results/
```

### 9.3 Per-run directory layout

Each Phase 1 run produces a self-contained directory:

```
results/
+-- run_<timestamp>_surf<surface>_npart<N>_..._lam<lambda>_seed<seed>/
|   |-- experiment.yaml          # verbatim copy of the input config
|   |-- solution/
|   |   |-- surface_part<N>_..._<timestamp>.h5    # Phase 1 output
|   |   +-- metadata.yaml
|   |-- traces/                  # PGD internal data per refinement level
|   |-- refinement/
|   |   |-- slsqp_btol0.001/      # one subdirectory per Phase 2 campaign
|   |   |   |-- iteration_001_<timestamp>.h5
|   |   |   |-- refinement.yaml   # config snapshot for reproducibility
|   |   |   +-- refinement.log
|   |   +-- trust-constr_btol0.001_lbfgs30/
|   |   +-- ...
|   |-- analysis/                # energy and constraint plots
+-- logs/
|   +-- relaxation.log
```

### 9.4 Experiment directory layout (parameter sweeps)

Sweep runs are grouped by surface and partition count:

```
results/
+-- torus_npart10/
```

```

|-- experiment_index.yaml           # auto-maintained index of all runs
|-- run_<timestamp>_..._lam0.5_seed42/
|   +-- ...                       # same per-run layout as above
|-- run_<timestamp>_..._lam1.0_seed42/
|   +-- ...
|-- sweeps/                       # sweep provenance records
|   |-- <timestamp>_<sweep_name>.yaml
|   +-- <timestamp>_<sweep_name>_summary.csv
+-- analysis/                     # experiment-wide comparison plots
    |-- heatmap_perimeter.png
    |-- line_perimeter.png
    +-- convergence_overlay.png

```

The `experiment_index.yaml` file is the central record. It lists every run with its parameters, status, and key metrics (perimeter, final energy, mesh size, convergence flag). Perimeter is the primary comparison metric because it is resolution-independent.

## 10 Managing Disk Space

The project directory `/proj/snic2020-15-36/private/` has a finite quota that is **shared by all collaborators**. A parameter sweep with 20 combinations can easily generate several gigabytes of HDF5 solution files and PGD trace data. Check usage regularly and prune low-quality runs after each sweep.

### 10.1 Checking current disk usage

```

# Total usage of the shared results directory
du -sh /proj/snic2020-15-36/private/LINKED_LST_MANIFOLD/results/

# Usage broken down by experiment directory
du -sh /proj/snic2020-15-36/private/LINKED_LST_MANIFOLD/results/*/

# Usage per individual run inside one experiment
du -sh /proj/snic2020-15-36/private/LINKED_LST_MANIFOLD/results/torus_npart10/run_*/

# Show your project allocation and core-hour balance
projinfo

```

### 10.2 Pruning runs with `cleanup_sweep_results.py`

`cluster/cleanup_sweep_results.py` deletes run directories from a sweep experiment, keeping only the  $N$  best runs ranked by perimeter (or another metric). It reads `experiment_index.yaml` to rank runs and then removes the rest.

#### Important

**Always run a dry-run preview first.** Without `-execute` the script only prints what *would* be deleted — no files are touched. Only add `-execute` once you have reviewed the output and are satisfied with the selection.

**Safe items:** the `sweeps/` provenance directory is never touched. A `cleanup_log.yaml` file is written (or appended) in the experiment directory with full provenance of every deletion so you can always audit what was removed and why. `experiment_index.yaml` is *not* modified; rebuild it by running `-mode collect` after cleanup.



### 10.2.1 Typical workflow

```
# 1. Check how much space the experiment is using
du -sh /proj/snic2020-15-36/private/LINKED_LST_MANIFOLD/results/torus_npart10/

# 2. Preview what would be deleted (no changes made)
python cluster/cleanup_sweep_results.py \
    --experiment-dir \
    /proj/snic2020-15-36/private/LINKED_LST_MANIFOLD/results/torus_npart10/

# 3. Execute: delete all but the 10 best runs by perimeter (default)
python cluster/cleanup_sweep_results.py \
    --experiment-dir \
    /proj/snic2020-15-36/private/LINKED_LST_MANIFOLD/results/torus_npart10/ \
    --execute

# 4. Rebuild the experiment index from surviving directories
python sweep/parameter_sweep.py \
    --sweep sweep/parameters/sweep_torus_lambda.yaml \
    --mode collect \
    --output-dir /proj/snic2020-15-36/private/LINKED_LST_MANIFOLD/results
```

### 10.2.2 Available options

| Option                                     | Default    | Description                                                        |
|--------------------------------------------|------------|--------------------------------------------------------------------|
| <code>--experiment-dir &lt;path&gt;</code> | (required) | Experiment directory containing <code>experiment_index.yaml</code> |
| <code>--keep &lt;N&gt;</code>              | 10         | Number of best runs to keep                                        |
| <code>--metric &lt;name&gt;</code>         | perimeter  | Metric to rank by (lower = better)                                 |
| <code>--execute</code>                     | —          | Actually delete. Without this flag, only a preview is shown.       |

### 10.2.3 Further examples

Keep only the 5 best runs:

```
python cluster/cleanup_sweep_results.py \
    --experiment-dir /proj/.../results/torus_npart10/ \
    --keep 5 --execute
```

Rank by `final_energy` instead of `perimeter`:

```
python cluster/cleanup_sweep_results.py \
    --experiment-dir /proj/.../results/torus_npart10/ \
    --metric final_energy --execute
```

#### Note

The dry run also estimates how much disk space would be freed, broken down per run directory. Use this estimate to decide how many runs to keep before committing to a deletion.

## 11 Retrieving and Analysing Results

### 11.1 Transferring files to your local machine

Use `rsync` to copy results efficiently (only transfers changed files):

```
# Copy a single run directory
```

```
rsync -avz --progress \
    <your_username>@pelle.uppmax.uu.se:/proj/snic2020-15-36/private/ \
    LINKED_LST_MANIFOLD/results/run_<your_run>/ \
    ~/local-results/run_<your_run>/

# Copy an entire experiment directory (e.g. torus_npart10)
rsync -avz --progress \
    <your_username>@pelle.uppmax.uu.se:/proj/snic2020-15-36/private/ \
    LINKED_LST_MANIFOLD/results/torus_npart10/ \
    ~/local-results/torus_npart10/

# Transfer only solution HDF5 files (skip large trace data)
rsync -avz --progress --include="*.h5" --include="*/" \
    --exclude="traces/**" \
    <your_username>@pelle.uppmax.uu.se:/proj/snic2020-15-36/private/ \
    LINKED_LST_MANIFOLD/results/run_<your_run>/ \
    ~/local-results/run_<your_run>/
```

#### Note

`rsync` preserves directory structure and skips files that have not changed since the last transfer. Run it again at any time to pull updates without re-downloading everything.

## 11.2 Local analysis (on your laptop)

Once the files are on your local machine, activate your local environment and run the analysis scripts from the repository root:

```
# Activate pyenv environment (see .python-version)
pyenv activate surface-partition

# Analyse a single run
python scripts/optimization_analyzer.py \
    --results-dir ~/local-results/run_<your_run>/

# Sweep-wide analysis (reads experiment_index.yaml)
python sweep/sweep_analyzer.py \
    --experiment-dir ~/local-results/torus_npart10/

# Timing analysis (only for runs with --profile flag)
python sweep/timing_analyzer.py \
    --experiment-dir ~/local-results/torus_npart10/
```

## 11.3 Visualising partitions (requires PyVista)

Visualisation scripts require a local PyVista install (`pip install -e ".[viz]"`). On the cluster they can only produce saved images (no interactive window).

```
# Fast production viewer (recommended for fine meshes)
python scripts/visualize_partition_fast.py \
    --solution ~/local-results/run_.../solution/surface_...h5

# Visualise a Phase 2 refined checkpoint
python scripts/visualize_partition_fast.py \
    --solution ~/local-results/run_.../refinement/slsqp_bt010.001/ \
    iteration_005_<timestamp>.h5 \
    --show-steiner
```

```
# Save an image instead of opening an interactive window
python scripts/visualize_partition_fast.py \
    --solution ... --save partition.png
```

## 12 Useful SLURM Commands

| Command                                                           | What it does                                      |
|-------------------------------------------------------------------|---------------------------------------------------|
| <code>squeue -u &lt;username&gt;</code>                           | List your running and pending jobs                |
| <code>squeue -u &lt;username&gt; -t R</code>                      | List only running jobs                            |
| <code>scancel &lt;job_id&gt;</code>                               | Cancel a job                                      |
| <code>scancel -u &lt;username&gt;</code>                          | Cancel <i>all</i> your jobs                       |
| <code>sacct -j &lt;job_id&gt; -format=Elapsed,MaxRSS,State</code> | Job runtime, memory, and exit state               |
| <code>jobinfo &lt;job_id&gt;</code>                               | UPPMAX-specific detailed job info                 |
| <code>projinfo</code>                                             | Show your allocation usage (core-hours + storage) |
| <code>uquota</code>                                               | Show your home directory disk usage               |
| <code>du -sh &lt;dir&gt;</code>                                   | Summarise disk usage of a directory               |
| <code>du -sh &lt;dir&gt;/*/</code>                                | Per-subdirectory usage breakdown                  |

## 13 Troubleshooting

**“Virtual environment not found” at job startup.** The job cannot find `VENV_DIR`. Check that you:

- Updated `VENV_DIR` in `cluster/pelle_config.sh` to your own username (Section 4.7).
- Ran the setup steps in Section 4 on Pelle (not just locally).
- Did not delete or move the `venv` directory.

**“ModuleNotFoundError: No module named ...” in the job log.** The Python module was loaded but the package is not in the `venv`. Re-run the install step:

```
module load Python/3.11.5-GCCcore-13.3.0
source ~/venvs/surface-partition/bin/activate
cd ~/projects/surface-partition
pip install -e ".[viz,implicit]"
```

**Job fails immediately with exit code 1.** Check the `.err` log file in `slurm_logs/`. Common causes:

- Config YAML file path not found — use an absolute path or ensure the path is correct relative to `PROJECT_BASE`.
- HDF5 solution file not found — check the `-solution` path.
- Permission denied on `/proj/snic2020-15-36/` — confirm with the PI that you have been added to the storage project.

**“Permission denied” writing to the results directory.** You need to be a member of the storage allocation `snic2020-15-36`. Ask the PI to add you via SUPR.

**The job is queued for a long time.** Check allocation usage with `projinfo`. If the core-hour balance is exhausted the queue priority drops. Reduce `-cpus` and `-time` to the minimum needed for your run.

**I cannot find my results after the job finishes.** Results are written to `PROJECT_BASE/results/`, which is `/proj/snic2020-15-36/private/LINKED_LST_MANIFOLD/results/` — not to your home directory. Use `ls` there and look for a `run_<timestamp>_...` directory with a recent timestamp.

## A pelle\_config.sh Quick Reference

The table below lists every variable in `cluster/pelle_config.sh` and whether collaborators need to edit it.

| Variable      | Edit?      | Description                                |
|---------------|------------|--------------------------------------------|
| PROJECT_ID    | No         | SLURM allocation ID                        |
| PROJECT_BASE  | No         | Shared data root on <code>/proj/</code>    |
| RESULTS_BASE  | No         | Derived from <code>PROJECT_BASE</code>     |
| REPO_DIR      | <b>Yes</b> | Absolute path to your clone of the repo    |
| PYTHON_MODULE | No*        | Module string for <code>module load</code> |
| VENV_DIR      | <b>Yes</b> | Absolute path to your Python venv          |
| DEFAULT_TIME  | No         | Default SLURM time limit                   |
| DEFAULT_CPUS  | No         | Default CPU count                          |
| DEFAULT_MEM   | No         | Default memory per job                     |

\*Only edit `PYTHON_MODULE` if the module name changes on UPPMAX, e.g. after a system upgrade.

## B Available Experiment Configurations

The `parameters/` directory ships with four ready-to-use experiment configs:

| Config file                               | Surface                             | Partitions |
|-------------------------------------------|-------------------------------------|------------|
| <code>torus_10part.yaml</code>            | Torus ( $R = 1$ , $r = 0.6$ )       | 10         |
| <code>ellipsoid_6part.yaml</code>         | Ellipsoid                           | 6          |
| <code>double_torus_10part.yaml</code>     | Double torus (implicit)             | 10         |
| <code>banchoff_chmutov_12part.yaml</code> | Banchoff-Chmutov order 4 (implicit) | 12         |

The double-torus and Banchoff-Chmutov configs use implicit surface meshing (marching cubes) and require the `.[implicit]` dependency group, which is included in the cluster installation above.